NumiVault 2.0

Product Requirements Document (PRD).

1. Résumé du Projet et Vision

Vision : Transformer NumiVault, d'un outil de gestion personnel nécessitant des saisies manuelles, en une plateforme de référence (SaaS) automatisée et communautaire pour les numismates et investisseurs.

Objectif principal : Permettre à tout utilisateur de gérer sa collection de pièces, de suivre la valeur de ses investissements en temps réel (cours des métaux) et de gérer ses ventes à travers une interface fluide, sans avoir à créer manuellement les fiches techniques des pièces. L'application s'appuiera sur une base de données mondiale partagée (façon "Vivino" pour le vin ou "Discogs" pour la musique).

2. Personas (Public Cible)

1. **Le Collectionneur Passionné :** Veut cataloguer ses pièces rapidement, voir de belles photos, et connaître la complétion de sa collection par année/pays.

2. **L'Investisseur (Stacker) :** S'intéresse principalement au poids en métal (Or/Argent), à la pureté, au prix d'achat, et au "Spot Value" en temps réel pour calculer son P/L (Profit/Loss).

3. **Le Vendeur/Marchand Amateur :** Achète des lots, revend à l'unité. A besoin d'un CRM (gestion des acheteurs) fluide pour suivre ses marges, ses clients réguliers (ex: "Ricardo") et éditer des factures ou des listes de vente.

3. Problématiques actuelles à résoudre (Pain Points)

- **Saisie manuelle chronophage :** Devoir entrer le poids, le titrage (pureté) et la photo pour chaque nouvelle pièce.
- **Évaluation obsolète des métaux :** Les prix des métaux précieux ne se mettent pas à jour automatiquement.
- **UX/UI rigide :** La navigation entre la vue globale de la collection, les détails d'une pièce et la gestion des ventes n'est pas assez fluide.
- **CRM déconnecté :** La gestion des acheteurs et des ventes ressemble trop à un tableau Excel complexe.

4. Périmètre et Fonctionnalités (Scope)

4.1. Base de données globale (Le "Core" de la V2)

- **Catalogue Mutualisé (Master DB) :** Création d'une base de données centrale des pièces (Pays, Année, Valeur faciale, Poids, Titrage, Métal, Photos avers/revers).
- **Intégration API Numista :** Utiliser l'API de Numista (ou un import structuré de catalogues) pour pré-remplir cette Master DB.
- **Ajout en 1 clic :** L'utilisateur cherche "20 Francs Vreneli" ou "5 Francs Léopold II" et l'ajoute à sa collection en précisant uniquement *son* prix d'achat, *son* état (Grade) et *sa* date d'achat.

4.2. Automatisation Financière

- **API Métaux Précieux :** Intégration d'une API tierce (ex: *Metals-API*, *GoldAPI.io* ou *MetalpriceAPI*) pour récupérer les cours de l'Or, l'Argent, le Platine et le Palladium deux fois par jour.
- **Calcul automatique du "Bullion Value" :** Dès que le Spot Price change, le tableau de bord met à jour le P/L latent de toute la collection instantanément, sans action de l'utilisateur.

4.3. Amélioration de l'UX/UI (Interface)

- **Vue Collection Repensée :**
    - *Mode Galerie :* De grandes cartes très visuelles (idéal pour mobile/tablette) mettant en valeur la beauté des pièces.
    - *Mode Liste (Tableur) :* Pour les investisseurs qui veulent voir des données pures (Poids, Spot, P/L).
- **Parcours d'ajout unifié :** Un bouton global "Ajouter une pièce" qui ouvre une modale de recherche intelligente (auto-complétion basée sur le catalogue global).

4.4. Gestion des Ventes et CRM (Clients)

- **Fiches Acheteurs :** Un véritable mini-CRM. En cliquant sur "Anastassia Kistler", on voit toutes les pièces qu'elle a achetées, le profit généré avec elle, et ses préférences.
- **Flux de vente fluide :** Glisser-déposer (Drag & Drop) d'une pièce de la "Collection" vers "À Vendre" ou "Vendu". Génération automatique du calcul de marge (Markup).

5. Exigences Techniques (Tech Stack & Architecture)

- **Architecture SaaS (Multi-tenant) :** Chaque utilisateur doit avoir son propre espace (User_ID relié à ses pièces), mais tous pointent vers la même base de données de "Modèles de pièces" pour éviter la duplication.
- **APIs Externes requises :**
    - *Données Numismatiques :* API Numista (ou web scraping légal conditionné pour créer la base initiale).
    - *Métaux Précieux :* GoldAPI (souvent gratuit jusqu'à 100 requêtes/mois, parfait pour des updates quotidiens) ou currencylayer / fixer.io si couplé à des devises.
    - *Devises :* Convertisseur CHF, EUR, USD en temps réel.
- **Base de données :** Une structure relationnelle stricte (PostgreSQL par exemple) :
    - Table Coin_Master (Les modèles fixes : 1 Franc Helvetia 1899).
    - Table User_Collection (Les exemplaires possédés : le 1 Franc de l'utilisateur X, état VF-30, acheté à 10 CHF).

6. Feuille de route (Roadmap d'implémentation)

- **Phase 1 : Résolution des bloqueurs techniques**
    - Connecter et tester une API pour les prix de l'Or/Argent (Spot Prices en direct).
    - Séparer la base de données en deux : "Catalogue Global" vs "Inventaire Utilisateur".
- **Phase 2 : Importation des données & Recherche**
    - Brancher l'API Numista pour permettre la recherche globale.
    - Créer le nouveau flux "Ajouter une pièce" basé sur la recherche.
- **Phase 3 : Refonte UI et Fluidité**
- **Améliorer le Dashboard (tes graphiques actuels sont très bien, il faut juste les rendre plus interactifs).**
    - Créer les vues "Galerie" et "Tableau" pour la collection.
- **Phase 4 : Module CRM & Ventes**
    - Lier les acheteurs aux transactions de manière plus visuelle.

- Créer des profils acheteurs dédiés.

SCHEMA DE BASE DE DONNEES

C'est parti. Pour passer d'une application "statique" à une plateforme évolutive et automatisée, nous allons utiliser une structure de base de données relationnelle (comme PostgreSQL).

L'idée centrale est de séparer ce qui est **immuable** (une pièce de 1 Franc de 1899 pèse toujours 5g et contient 83.5% d'argent) de ce qui est **propre à l'utilisateur** (son prix d'achat, l'état de sa pièce, sa photo).

Voici la proposition de schéma pour **NumiVault V2** :

1. Le Catalogue Global (Master_Coins)

Cette table est la "bibliothèque". Elle est partagée par tous les utilisateurs. Si elle est bien remplie (via API Numista), l'utilisateur n'a plus qu'à "chercher et cliquer".

Champ	Type	Description
id	UUID / INT	Clé primaire unique.
numista_id	INT	ID de référence pour les futures mises à jour via API.
country_code	VARCHAR(2)	Code ISO (CH, FR, US...).
title	VARCHAR	Nom de la pièce (ex: "5 Francs - Léopold II").
year	INT	Année d'émission.
metal_type	ENUM	Gold, Silver, Platinum, Copper, etc.
purity	DECIMAL	Pureté du métal (ex: 0.835, 0.900).
weight_gross	DECIMAL	Poids total en grammes (ex: 5.00).
weight_net	DECIMAL	Poids de métal fin (calculé : Gross * Purity).
image_obverse_url	TEXT	URL de l'image officielle (avers).
image_reverse_url	TEXT	URL de l'image officielle (revers).

2. L'Inventaire Utilisateur (User_Inventory)

C'est ici que l'utilisateur stocke ses exemplaires personnels. Chaque ligne pointe vers une pièce du catalogue global.

Champ	Type	Description
id	UUID	Clé primaire.
user_id	UUID	Lien vers la table Users (Propriétaire).

coin_master_id	UUID	**FK :** Lien vers la fiche technique dans Master_Coins.
grade	VARCHAR	État de la pièce (MS-63, AU-58, VF, etc.).
purchase_price	DECIMAL	Prix payé (dans la devise de l'utilisateur).
purchase_date	DATE	Date d'acquisition.
status	ENUM	In Collection', 'Listed for Sale', 'Sold'.
personal_notes	TEXT	Remarques privées de l'utilisateur.
user_image_url	TEXT	Photo spécifique de *sa* pièce (optionnel).

3. CRM : Acheteurs & Ventes (Buyers & Sales)

Pour rendre la gestion des ventes fluide, on sépare les contacts des transactions.

Table Buyers :

- id, user_id (le vendeur), name, email, platform (Ricardo, eBay, Vente directe), total_spent (calculé).

Table Sales :

- id
- inventory_id (FK vers User_Inventory)
- buyer_id (FK vers Buyers)
- sale_price (Prix de vente final)
- sale_date
- shipping_cost
- platform_fees (Frais de commission)

4. Automatisation des Prix (Spot_Prices)

Une table simple mise à jour par ton API (GoldAPI par exemple) toutes les quelques heures.

Métal	Price_per_gram	Currency	Last_Update
Gold	72.45	CHF	2026-04-25 22:00
Silver	0.88	CHF	2026-04-25 22:00

Pourquoi cette structure change tout ?

1. **Calcul du Profit en temps réel :** Pour chaque pièce en stock : (Master_Coins.weight_net * Spot_Prices.Price_per_gram) - User_Inventory.purchase_price.

2. **Partage communautaire :** Si un utilisateur crée une fiche complète pour une pièce rare qui n'est pas dans ton catalogue, tu peux la rendre "publique" pour que les autres puissent l'utiliser.

3. **Analyse CRM :** Tu peux instantanément voir quel acheteur te génère le plus de marge ou quelle plateforme (Ricardo vs eBay) est la plus rentable après déduction des platform_fees.

Voici le script SQL complet (format PostgreSQL) pour mettre en place cette nouvelle architecture. Ce script définit les tables, les relations (clés étrangères) et les contraintes nécessaires pour garantir l'intégrité de tes données.

Script SQL : Structure NumiVault V2

SQL

```
-- Extension pour la génération automatique d'UUID (Identifiants Uniques)
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 1. Types ENUM pour standardiser les saisies
CREATE TYPE metal_category AS ENUM ('Gold', 'Silver', 'Platinum', 'Palladium',
'Copper', 'Nickel', 'Other');
CREATE TYPE inventory_status AS ENUM ('In Collection', 'Listed for Sale', 'Sold');

-- 2. Table : Catalogue Global (Master_Coins)
-- Partagée entre tous les utilisateurs
CREATE TABLE master_coins (
   id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
   numista_id INTEGER UNIQUE, -- Pour synchronisation API
   title VARCHAR(255) NOT NULL,
   country_code VARCHAR(2) NOT NULL, -- ISO 3166-1 alpha-2
   year_issued INTEGER,
   metal metal_category NOT NULL,
   purity DECIMAL(5, 4) NOT NULL, -- ex: 0.9000
   weight_gross DECIMAL(10, 3) NOT NULL, -- en grammes
   weight_net DECIMAL(10, 3) GENERATED ALWAYS AS (weight_gross * purity)
STORED,
   image_obverse_url TEXT,
   image_reverse_url TEXT,
   created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 3. Table : Utilisateurs (Simplifiée pour l'exemple)
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    username VARCHAR(100) UNIQUE NOT NULL,
    base_currency VARCHAR(3) DEFAULT 'CHF'
);

-- 4. Table : Inventaire Utilisateur (User_Inventory)
CREATE TABLE user_inventory (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    master_coin_id UUID REFERENCES master_coins(id),
    grade VARCHAR(20), -- ex: MS-63, AU-58
    purchase_price DECIMAL(15, 2),
    purchase_date DATE,
    status inventory_status DEFAULT 'In Collection',
    personal_notes TEXT,
    custom_image_url TEXT, -- Si l'utilisateur veut uploader sa propre photo
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 5. Table : CRM Acheteurs (Buyers)
CREATE TABLE buyers (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    full_name VARCHAR(255) NOT NULL,
    email VARCHAR(255),
    platform VARCHAR(100), -- ex: 'Ricardo', 'eBay', 'Direct'
    notes TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 6. Table : Ventes (Sales)
CREATE TABLE sales (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    inventory_id UUID UNIQUE REFERENCES user_inventory(id) ON DELETE
CASCADE,
    buyer_id UUID REFERENCES buyers(id),
    sale_price DECIMAL(15, 2) NOT NULL,
    sale_date DATE DEFAULT CURRENT_DATE,
    shipping_cost DECIMAL(10, 2) DEFAULT 0,
    platform_fees DECIMAL(10, 2) DEFAULT 0,
    net_profit DECIMAL(15, 2) -- Peut être calculé via un trigger ou à la volée
);

-- 7. Table : Cours des Métaux (Spot_Prices)
CREATE TABLE spot_prices (
    metal metal_category PRIMARY KEY,
    price_per_gram DECIMAL(15, 4) NOT NULL,
```

```
    currency VARCHAR(3) DEFAULT 'CHF',
    last_updated TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

Points clés de cette structure :

- **master_coins.weight_net** : J'ai utilisé un champ calculé (GENERATED ALWAYS). La base de données calcule automatiquement le poids de métal fin dès que tu entres le poids total et la pureté.
- **user_inventory.master_coin_id** : C'est le lien vital. Plusieurs utilisateurs peuvent posséder le même modèle de pièce (même ID Master), mais chacun aura sa propre ligne dans user_inventory avec son propre prix d'achat.
- **sales.inventory_id** : La contrainte UNIQUE garantit qu'une pièce de ton inventaire ne peut pas être vendue deux fois.
- **Indépendance du CRM** : En séparant buyers de sales, tu peux suivre l'historique d'un client (ex: "Combien d'argent cet acheteur m'a-t-il rapporté au total sur l'année ?") très facilement.

La logique de l'API de recherche

Le flux idéal pour ton utilisateur serait :

1. Il tape "**20 francs vreneli**" dans une barre de recherche.
2. Ton application regarde d'abord dans master_coins (ton catalogue local).
3. Si elle ne trouve rien, elle interroge l'**API Numista**.
4. Si Numista renvoie un résultat, l'application crée automatiquement une ligne dans master_coins et l'affiche à l'utilisateur pour qu'il puisse l'ajouter à son inventaire.

L'algorithme de l'API de recherche

Voici le code commenté qui gère ce fameux "pont" :

JavaScript

```
const express = require('express');
const axios = require('axios'); // Pour faire la requête
vers Numista
const db = require('./db'); // Ta connexion à la base
PostgreSQL
```

```
const router = express.Router();

// Route: GET /api/search?q=20+francs
router.get('/search', async (req, res) => {
    const searchQuery = req.query.q;

    if (!searchQuery || searchQuery.length < 3) {
        return res.status(400).json({ error: "Veuillez
entrer au moins 3 caractères." });
    }

    try {
        // ÉTAPE 1 : Chercher dans notre propre base
(master_coins) d'abord
        // On utilise ILIKE pour une recherche insensible
à la casse
        const localResults = await db.query(
            `SELECT * FROM master_coins WHERE title ILIKE
$1 OR country_code ILIKE $1`,
            [`%${searchQuery}%`]
        );

        // Si on a des résultats locaux pertinents, on
s'arrête là !
        if (localResults.rows.length > 0) {
            return res.json({
                source: 'local',
                data: localResults.rows
            });
        }

        // ÉTAPE 2 : Rien trouvé en local ? On interroge
l'API Numista
        // (Note: Il te faudra une clé API Numista)
        const numistaResponse = await axios.get(`https://
api.numista.com/api/v3/coins`, {
            params: { q: searchQuery },
            headers: { 'Numista-API-Key':
process.env.NUMISTA_API_KEY }
        });

        const numistaCoins = numistaResponse.data.items;
```

```
        if (!numistaCoins || numistaCoins.length === 0) {
            return res.json({ source: 'none', data:
[] }); // Rien trouvé nulle part
        }

        // ÉTAPE 3 : On sauvegarde les nouveaux résultats
dans NOTRE base
        const newlySavedCoins = [];

        for (const coin of numistaCoins) {
            // On vérifie d'abord que ce numista_id n'a
pas été ajouté entre-temps
            const exists = await db.query(`SELECT id FROM
master_coins WHERE numista_id = $1`, [coin.id]);

            if (exists.rows.length === 0) {
                // On map (traduit) les données de
Numista vers notre format SQL
                const insertQuery = `
                    INSERT INTO master_coins (
                        numista_id, title, country_code,
year_issued, metal, purity, weight_gross
                    ) VALUES ($1, $2, $3, $4, $5, $6, $7)
                    RETURNING *;
                `;

                const values = [
                    coin.id,
                    coin.title,
                    coin.issuer.code, // Dépend de la
structure JSON de Numista
                    coin.min_year,
                    mapMetal(coin.composition.metal), //
Fonction utilitaire pour matcher ton ENUM
                    coin.composition.purity || 1.0,
                    coin.weight
                ];

                const saved = await db.query(insertQuery,
values);
                newlySavedCoins.push(saved.rows[0]);
            }
        }
```

```
        // ÉTAPE 4 : On renvoie les données fraîchement
enregistrées à l'utilisateur
        return res.json({
            source: 'numista_imported',
            data: newlySavedCoins
        });

    } catch (error) {
        console.error("Erreur lors de la recherche:",
error);
        res.status(500).json({ error: "Erreur
serveur" });
    }
});

module.exports = router;
```

💡 **3 Points d'attention cruciaux pour que ça marche bien :**

1. **Le Mapping des données (`mapMetal`) :** Les API externes n'utilisent jamais exactement les mêmes termes que toi. Numista pourrait renvoyer `"Silver 835"` alors que ton ENUM de base de données attend `'Silver'` avec une colonne `purity`à `0.835`. Tu devras créer une petite fonction utilitaire pour nettoyer et adapter ces données avant de faire ton `INSERT INTO`.

2. **Les Limites de l'API (Rate Limiting) :** L'API de Numista a sûrement une limite de requêtes par minute/jour. C'est pourquoi **l'étape 1 (vérifier ta base) est obligatoire**. Cela protège ton quota API.

3. **Le Debounce sur le Frontend (Côté Client) :** Dans ton application React/Vue, si l'utilisateur tape "Vreneli", il ne faut pas envoyer une requête à chaque lettre tapée (V... Vr... Vre...). Il faut attendre que l'utilisateur arrête de taper pendant environ 300 à 500 millisecondes (c'est ce qu'on appelle un *debounce*) avant d'appeler ta route `/api/search`.

Avec cette logique, plus tes utilisateurs vont chercher de pièces, plus ton catalogue `master_coins` va s'enrichir automatiquement, et plus ton application deviendra rapide et autonome.

DASHBOARD STATISTIQUES ET P/L

1. Les Métriques Clés (KPIs) à afficher

Voici les blocs de statistiques (Cartes) qui doivent figurer en haut du Dashboard.

A. Vue "Investisseur" (Le P/L et les Métaux)

C'est ici que l'automatisation des prix de l'or et de l'argent prend tout son sens.

- **Total Investi (Cost Basis) :** La somme exacte de ce que l'utilisateur a dépensé pour les pièces actuellement en sa possession.
- **Valeur Métal Actuelle (Bullion Value) :** La valeur de la collection basée *uniquement* sur le poids des métaux au cours du jour.
- **P/L Latent (Unrealized Profit/Loss) :** Le bénéfice ou la perte potentielle si l'utilisateur fondait sa collection aujourd'hui. (Affiché en devise absolue + pourcentage).
- **P/L Réalisé (Realized Profit/Loss) :** Le bénéfice net généré par les pièces déjà vendues.

B. Vue "Collectionneur" (L'Inventaire)

- **Total des Pièces :** Nombre d'exemplaires dans la collection.
- **Diversité :** Nombre de pays différents, nombre d'années distinctes.
- **Distribution par Métal :** (ex: 60% Argent, 30% Or, 10% Cuivre).

C. Vue "Vendeur" (Performances Commerciales)

- **Chiffre d'Affaires (Total Revenue) :** Somme totale des ventes.
- **Marge Moyenne (Average Markup) :** Le pourcentage de profit moyen par pièce vendue.
- **Frais Totaux :** Somme des frais de port et commissions (Ricardo, eBay) payés.

2. La Logique Mathématique (Le "Moteur" P/L)

Pour que ton développeur (ou toi-même) comprenne comment calculer cela avec la base de données que nous avons créée, voici les formules logiques à implémenter dans le backend :

Calcul du P/L Latent (Unrealized P/L) pour une pièce :

1. Trouver le poids net en métal : `master_coins.weight_net`
2. Trouver le prix au gramme du jour : `spot_prices.price_per_gram` (pour ce métal spécifique)
3. Calculer la valeur Spot : `(weight_net * price_per_gram)`

4. Soustraire le prix d'achat : `Valeur Spot - user_inventory.purchase_price`

Calcul du Profit Net Réalisé (Realized P/L) sur une vente : La formule doit absolument prendre en compte les frais pour être réaliste.

```
Profit Net = sales.sale_price - user_inventory.purchase_price - sales.shipping_cost - sales.platform_fees
```

Note technique : Pour optimiser les performances de ton application, le backend ne doit pas recalculer le poids net de l'or de toute la collection à chaque rechargement de page. Il vaut mieux stocker ces totaux globaux en cache (Redis) et les mettre à jour uniquement quand les prix `spot_prices` changent (ex: toutes les 4 heures).

3. Recommandations UX/UI pour les Graphiques

Les chiffres bruts ne suffisent pas, il faut les visualiser. Voici les graphiques essentiels à intégrer :

- **Graphique en ligne (Line Chart) : "Évolution de la Valeur"**
  - *Axe X :* Temps (Mois/Années).
  - *Axe Y :* Valeur en CHF.
  - *Séries (2 lignes) :* Ligne 1 (Total Investi, souvent "en escalier" lors des achats) VS Ligne 2 (Valeur Métal, qui fluctue avec les marchés). L'écart entre les deux montre visuellement le profit.
- **Graphique circulaire (Donut Chart) : "Répartition par Pays & Métal"**
  - Comme sur tes captures d'écran, c'est parfait pour voir en un clin d'œil que l'on possède 90% de pièces suisses en argent.
- **Graphique en barres (Bar Chart) : "Ventes Mensuelles"**
  - Affiche le volume de vente par mois, avec une couleur pour le coût d'achat et une autre par-dessus pour le profit net généré.

Avec ça, ton PRD définit non seulement *ce qu'il faut afficher*, mais aussi *comment le calculer de manière juste* (en incluant les frais des plateformes, un point souvent oublié par les collectionneurs !).

MODULE MINI-CRM

1. Objectifs du Mini-CRM

L'objectif n'est pas de recréer Salesforce, mais de fournir un outil ultra-ciblé pour :

- Centraliser les contacts des acheteurs (qu'ils viennent de Ricardo, eBay, ou du bouche-à-oreille).
- Calculer la rentabilité réelle par canal de vente (en déduisant les frais de port et les commissions).
- Identifier les acheteurs récurrents pour leur proposer des pièces en direct (et économiser les frais de plateforme).

2. Fonctionnalités Clés (User Stories)

- **Le Profil Acheteur (Buyer Profile) :** * Chaque acheteur possède une "fiche" contenant son nom, son pseudo (ex: `AigleArgent99`), sa plateforme d'origine, et éventuellement son adresse email ou postale pour les envois.
    - *Vue "Historique" :* Sur cette fiche, je vois la liste de toutes les pièces que cette personne m'a achetées, la date, et la marge nette qu'elle m'a générée.
- **Le Flux de Vente Simplifié (Checkout Flow) :**
    - Dans la vue "Collection", je sélectionne une pièce et clique sur "Vendre".
    - Une fenêtre (modale) s'ouvre :
        1. *Qui est l'acheteur ?* (Recherche par auto-complétion dans la base CRM, ou bouton "Nouvel Acheteur").
        2. *Prix de vente convenu.*
        3. *Frais :* Coût d'expédition (ex: 5 CHF pour un recommandé) et frais de plateforme (ex: 12% sur Ricardo).
    - En validant, la pièce passe du statut "In Collection" à "Sold", et le Dashboard (P/L réalisé) se met à jour instantanément.
- **Analyse des Plateformes (Platform Analytics) :**
    - Un petit tableau de bord dédié aux ventes qui compare les canaux.
    - *Exemple :* "Ricardo : 80% de mes ventes, 15% de frais moyens" vs "Vente Directe : 20% de mes ventes, 0% de frais".

3. Interface Utilisateur (UX/UI) recommandée

Pour rendre cela intuitif, je te conseille de diviser cette partie en deux écrans principaux :

A. L'écran "Réseau / Mes Acheteurs"

- Un tableau simple (style DataGrid) listant tous les clients.

- **Colonnes recommandées :** Nom/Pseudo | Plateforme | Nombre de pièces achetées | Total Dépensé | Profit Généré.
- **Filtres :** Pouvoir trier par "Meilleurs clients" (ceux qui ont dépensé le plus).

B. L'écran "Registre des Ventes" (Sales Ledger)

- Un historique chronologique de toutes les transactions (très utile pour la comptabilité).
- **Colonnes recommandées :** Date | Pièce vendue | Acheteur | Prix Brut | Frais | Profit Net.
- **Exportation :** Un bouton "Exporter en CSV/Excel" est indispensable ici pour que l'utilisateur puisse faire ses déclarations ou ses propres analyses en fin d'année.

Résumé de la valeur ajoutée de ce CRM :

Avec cette fonctionnalité, ton application ne sert plus seulement à *regarder* ses pièces, elle devient un véritable **outil de gestion d'entreprise (ou de side-business)** pour les passionnés. Le fait de pouvoir lier instantanément un acheteur récurrent à une marge nette donne un avantage énorme à ton application par rapport à un simple fichier Excel.

ELEMENTS ADDITIONNELS

API Numista:

L’application ne devra en aucun cas dépasser 2000 requêtes par mois.

Quelques conseils pour la suite :

1. **Dès que tu auras ton API Key Numista :** Commence par le script de recherche, car c'est lui qui va nourrir ta base de données au fil de l'eau.
2. **Pour le Dashboard :** Utilise une bibliothèque comme `Recharts` ou `Chart.js` pour la partie graphique. Elles sont très simples à intégrer avec React ou Vue et rendront tes courbes de profits très professionnelles.
3. **Pour le CRM :** N'essaie pas d'en faire trop tout de suite. Commence par une simple liste de clients liés à tes ventes. L'analyse détaillée viendra naturellement dans un second temps.